Security Assessment Report

Target URL: http://localhost:8000

Scan Metadata

Date of Scan:	2026-07-13 09:24:17 UTC
Scan Duration:	230.44 seconds
Total Findings:	1

Vulnerability Severity Summary

CRITICAL	HIGH	MEDIUM	LOW	INFO
1	0	0	0	0

Executive Summary

During the scheduled security assessment, a comprehensive penetration test was conducted against the target environment at `http://localhost:8000` to evaluate its resilience against external threats. The assessment identified a single, critical-severity vulnerability, with no high- or medium-severity issues detected. Left unaddressed, this critical vulnerability poses an immediate risk of system compromise, potentially allowing unauthorized actors to bypass security controls and access sensitive resources. We strongly recommend immediate remediation of this finding to safeguard the application's integrity and mitigate operational risk.

Reconnaissance Details

Resolved Target IP:	127.0.0.1
Technologies Profile:	Nginx, React
Discovered Subdomains:	None

Detailed Security Findings

1. SQL Injection in /api/users - *CRITICAL Severity* [CVE-2023-SQLI]

Scanner Source: SQL Injection Scanner | **Confidence:** Low

Description:

The application is vulnerable to blind time-based SQL injection on the `/api/users/profile` endpoint via the `id` parameter. During testing, injecting a payload designed to trigger a database delay (`SLEEP(5)`) resulted in a response latency of over 5 seconds. This confirms that user input is directly concatenated into a SQL query. An attacker can exploit this to extract sensitive database schema information and data, bypass authentication, or potentially execute administrative commands.

Remediation Guidance:

1. Implement Parameterized Queries (Prepared Statements): Ensure all database queries treat user inputs as data rather than executable code. For example, in Node.js/pg, use: `db.query('SELECT * FROM users WHERE id = $1', [userId])` instead of string interpolation.
2. Enforce Strict Input Validation: Validate that the `id` parameter strictly conforms to expected formats (such as UUID or integer) before processing.
3. Apply the Principle of Least Privilege: Restrict the database user account used by the application to only the necessary read/write permissions required for normal operations.

Technical Evidence:

```
--- REQUEST ---
POST /api/users/profile
{
  "id": "1' OR (SELECT 1 FROM (SELECT(SLEEP(5)))a)--",
  "auth_token": "eyJhbGciOiJIUzI1NiI..."
}

--- RESPONSE ---
Response Time: 5012ms
HTTP/1.1 200 OK
Content-Type: application/json
```